AMERICAN UNIVERSITY OF PHNOM PENH

STUDY LOCALLY. LIVE GLOBALLY.

Smart Meal Planner

Web Application

Course: COSC 221 001 - Computer Science B

Prof. Firend Alan RASCH

Date: 20th April 2026

Group Members: Lou Leakhena (2024449) & Noun Leang Y (2025481)

Smart Meal Planner – Report

1. Problem Statement

In modern daily life, many people struggle to manage their diet properly. They often do not know how many calories they need or how to plan meals that match their personal goals. As a result, they may eat too much, too little, or choose unbalanced foods.

Even when users understand calorie intake, selecting the right combination of food is still difficult. Most people cannot easily decide which foods will give the best nutritional value within a limited calorie amount.

This project focuses on solving these problems by building a simple and practical system that helps users calculate their calorie needs and generate an optimized meal plan.

2. Proposed Solution

The solution is a web-based application called **Smart Meal Planner**, developed using Python and Flask.

The system allows users to:

- Input personal data (weight, height, age, gender, activity level)
- Calculate daily calorie needs
- Add food items with calories and protein
- Generate an optimized meal plan automatically

The system combines **scientific calculation** with **algorithmic optimization**, making it both useful and practical.

3. Idea and Benefits

Idea Behind the Project

The idea is to combine **calories calculation** and **computer algorithms** in one system. Instead of just showing calorie numbers, the system actively helps users make better decisions about their meals.

Benefits

- Helps users understand their daily calorie needs
- Provides better food combinations automatically
- Saves time in meal planning
- Easy to use and accessible
- Can be extended into a real-world application

4. Features of the System

- **Calorie Calculator**: Calculates daily calorie needs using user input
- **Meal Optimizer**: Uses an algorithm to select foods that maximize protein
- **Food Management:** Users can add, clear, and manage food items
- **Data Visualization**: Displays a pie chart of calorie distribution
- **Session Storage**: Temporarily stores user data during usage
- **JSON Storage**: Saves the last optimized meal plan
- **Responsive Design**: Works across different devices
- **Flash Messages**: Provides feedback for errors and actions

5. Technical Requirements

The application uses the following technologies:

- **Python:** core programming language
- **Flask**: web framework
- **HTML & CSS**: frontend design
- **Chart.js**: data visualization
- **Pandas**: data handling
- **JSON File**: data storage (last_optimized.json)

The system runs locally using a web browser.

6. System Design

The system follows a **client-server structure**:

Frontend

- HTML templates (Jinja2)
- CSS styling
- Chart.js for graphs

Backend

- Flask handles routes and logic
- Python processes calculations and optimization
- Session and JSON store data

System Flow

1. User enters input on the webpage
2. Flask receives the request
3. Backend processes data (calculation or optimization)

4. Results are passed to templates
5. Output is displayed to the user

Project Structure:

File / Folder	Purpose
main.py	App entry point
website/__init__.py	Flask app factory
website/views.py	Main routes: home and optimizer
website/auth.py	Auth routes: login and register
website/models.py	User, Food, Tracker, Optimizer classes
website/static/style.css	All styling and responsive design
website/templates/base.html	Shared layout, navbar, flash messages
website/templates/dashboard.html	Home page: calculator and pie chart
website/templates/optimizer.html	Meal optimizer page
website/templates/login.html	Login page
website/templates/register.html	Register page
website/static/style.css	Design of the web

7. Algorithms and Data Structures

7.1 Algorithm: 0/1 Knapsack

The system uses the **0/1 Knapsack algorithm** to optimize meal planning.

- Each food item has:
  - Calories (constraint)
  - Protein (value)

- Objective:
  Maximize total protein while staying within calorie limit

The algorithm builds a table to compare different combinations and selects the best result.

7.2 Data Structures Used

- **List**: stores food items
- **Dictionary**: groups data (e.g., meal types, chart data)

These structures help organize and process data efficiently.

8. Data Manipulation (Calorie Calculation)

The system uses the **Mifflin-St Jeor formula**:

- Male:
  $BMR = 10W + 6.25H - 5A + 5$
- Female:
  $BMR = 10W + 6.25H - 5A - 161$

Where:

- W = weight (kg)
- H = height (cm)
- A = age

After calculating BMR:

- Multiply by activity level
- Adjust based on user goal (lose, gain, maintain)

This produces the final daily calorie target.

9. Data Storage (JSON Usage)

The system uses a JSON file called **last_optimized.json** to store the most recent meal plan.

Purpose

- Save optimized results
- Allow data to persist after refresh
- Provide simple storage without database

How it Works

- After optimization, results are written to JSON
- When the page reloads, data is read from the file
- The file is updated each time a new plan is generated

Example Structure

```
{
  "foods": [
    {"name": "Chicken Breast", "calories": 200, "protein": 30},
    {"name": "Egg", "calories": 70, "protein": 6}
  ],
  "total_calories": 270,
  "total_protein": 36
}
```

10. Testing

The system was tested to ensure correct functionality:

Input Testing

- Valid input → correct calculation
- Invalid input → error message

Feature Testing

- Calorie calculation works correctly
- Food addition and clearing works
- Optimization produces correct results

All tests confirm that the system works reliably

11. Limitations

- Only considers calories and protein
- No permanent database
- Basic optimization model
- Authentication is not fully implemented

12. Reflection

This project helped improve skills in:

- Python programming
- Web development (Flask)
- Algorithm implementation

- Problem-solving

One challenge was understanding and solving the Knapsack problem by suing Dynamic Programming algorithm. Another challenge was connecting backend logic with frontend display.

13. Conclusion

The Smart Meal Planner is a simple but effective system that combines multiple advanced Python concepts. It helps users calculate calorie needs and generate optimized meal plans.

Overall, this project shows how programming can be used to solve real-world problems.

References

GeeksforGeeks. (2024). *0-1 Knapsack Problem*. https://www.geeksforgeeks.org/0-1-knapsack-problem-dp-10/

Medscape. (n.d.). *Mifflin-St Jeor equation calculator*. Retrieved April 22, 2026, from https://reference.medscape.com/calculator/846/mifflin-st-jeor-equation